Energy Efficient Data Centers with Task Distribution

Thuc X. Nguyen

January 17, 2023

West Seattle High School

3000 California Avenue Southwest,

Seattle, Washington 98116

Table of Contents

Acknowledgement of Assistance……...3

Abstract……..4

1. Introduction:……5

1.1 Data Centers Impact and Efficiency…….. 5

1.2 Load Balancing…….. 6

2. Methods and Results:……6

2.1 Stateless Task Distribution using a Randomized Model……6

2.2 Probability of r Tasks in a Server……...7

2.3 Experiment Setup……10

2.4 Probability a Server is Overloaded…….. 10

2.5 Probability a Server is Underloaded…… 12

2.6 Load Balancing with Two-Choice Model……13

Load Balancing with One Random Choice…… 14

Load Balancing with Two Random Choices……... 15

Comparison of One versus Two Choices……17

2.7 Consistent Hashing For Stateful Task Distribution…… 19

Consistent Hashing with Replicas……... 22

3. Conclusion……... 27

References 28

Acknowledgement of Assistance

I received assistance from my parents who have experience in math and doing research. They helped guide me what textbooks I should read and what techniques are needed in writing a research paper. This allowed me to understand how the STEM community structures research papers and what the standard is. I also got mini lessons on certain topics so that I would gain background knowledge. Often I would be given advice, but at the end, I was the one who was executing the final product through perseverance.

Abstract

To have an efficient data center, all servers must be utilized as close to maximum capacity, evenly. Load balancing is a fundamental method to evenly distribute tasks to servers. In this study, load balancing was modeled by how tasks were distributed on all servers in a data center, by examining stateful and stateless servers. Graphs of the loads for each task, variance, and tail distribution of maximum loads were compared during the experiments. For stateless servers, the random choice method and an improved version of the two-choice method was explored. After this experiment, the results showed that the two-choice model was a **10.37** times improvement over the random model. By spreading the load evenly, the two-choice method can reduce the number of servers, and could save **16.70%** of potential CO_2 emissions. Furthermore, consistent hashing, which was needed for stateful servers, was researched and implemented. This led to a **5.02** times improvement as server replicas increased. This study showed how consistent hashing is a reliable and effective method for stateful servers.

1. Introduction:

1.1 Data Centers Impact and Efficiency

Data centers have a big impact on the environment. They account for almost 1% of the global energy usage, and the energy was estimated at around 203 terawatt hours[1]. This was equal to 158,581,315 tons of CO_2 emitted every year, or roughly equivalent to operating 36.3 coal plants in a single year[4]. In the US, the Department of Defense, is the single largest energy consumer and owns 2 times more data centers (772) than any other agency[2]. Even though they started consolidating them in 2010, there are many ways to optimize data centers for improved efficiency such as load balancing, which optimizes the computational load to make servers run to maximum computational efficiency and preventing carbon dioxide emissions[3]. These emissions can be mitigated using energy efficient computing. Also, a data center uses around 20 acres of land[5]. This is a lot of land that could have been used for many other purposes. This means data centers need to be efficient and purposeful.

Researchers at Google are trying to shift customers to the cloud which will use fewer servers to maximize efficiency and utilization, which has led to 70-90% energy reduction[6,7]. This has led to 6 times more computing power, and two times more efficiency than a standard data center[6,8]. While Google explored how data centers use green energy (e.g. wind and solar)[6], this study

explored algorithms that allowed servers to be utilized evenly, hence leading to less servers for the same amount of computation.

1.2 Load Balancing

Data centers are efficient when all servers are utilized as close to the maximum capacity. This problem can be modeled as how loads are distributed on all servers in a data center. Load balancing is a fundamental step to distributing tasks to servers. The ultimate goal is to maximize server utilization evenly by shrinking the task load on overloaded servers, and increasing load on idle servers. Optimally, each and every servers' load is as **close** as possible to the total number of tasks divided by the number of servers (the optimal). Throughout this article, improved methods of load balancing through the lens of energy efficient computing were discussed.

2. Methods and Results:

2.1 Stateless Task Distribution using a Randomized Model

Representing servers receiving tasks through simulations is important for load balancing. One way this can be simulated is by using a balls and bins model[9]. One at a time, balls were thrown into bins uniformly, at random. Next, the simulation used a dictionary as the data structure, the key representing the bin index, and the value as the number of balls in that particular bin. Instead

of balls and bins, it was substituted for tasks and servers. This study also discussed the math involved in implementing the balls and bins model.

2.2 Probability of *r* Tasks in a Server

In this section, the probability of a certain server having exactly *r* tasks in it was derived. Let *n* denote the number of servers, and *m* denote the number of tasks.

Since the probability of one task in a given server is:

$$P(one\ task) = \frac{1}{n}$$

The probability a given server has exactly *r* tasks is:

$$P(r\ tasks) = (\frac{1}{n})^r$$

This can furthermore be represented by distributing *m* tasks into *n* servers as a Binomial distribution, and denoting it as a random variable *X*. The Binomial distribution represents a series of *m* Bernoulli trials, where each trial has a success rate *p*. A Bernoulli trial returns either a success (denoted by a 1) or a failure (denoted as a 0).

$$p = \frac{1}{n}$$

$$X \sim Bin(m, \frac{1}{n})$$

Using the Binomial probability mass function, the probability of *r* tasks across *n* servers can be found:

$$P(r\ tasks) = \binom{m}{r}(\frac{1}{n})^r \times (\frac{n-1}{n})^{m-r}$$

$$P(r\ tasks) = \frac{m!}{r!(m-r)!} \times \frac{1}{n^r} \times (\frac{n-1}{n})^{m-r}$$

$$P(r\ tasks) = \frac{1}{r!} \times e^{\frac{-m}{n}} \times (\frac{m}{n})^r$$

Then, replace $\frac{m}{n}$ by λ, which will represent the load of each server.

$$P(r\ tasks) = \frac{e^{-\lambda}\lambda^r}{r!}$$

By plugging in λ, this equation is exactly identical to the **Poisson** probability mass function. This means that the loads on servers can also be modeled using the Poisson distribution. If *X* equals the number of tasks in a server, X ~ Pois(λ).

Also, below is the expectation of servers that contain *r* tasks:

$$E(Servers\ with\ r\ tasks) = n \times P(r\ tasks)$$

Remember, the expectation of a Poisson random variable is λ:

$$E(Load\ per\ server) = \lambda$$

When a server is ***completely idle***, there are no assigned tasks. The probability of no tasks on a given server is:

$$P(0\ tasks) = 1 \times e^{-\lambda} \times 1$$

$$P(0\ tasks) = e^{-\lambda}$$

This shows that the amount of idle servers exponentially decays at a rate of λ.

2.3 Experiment Setup

Throughout the experiment, the default inputs were 1,000 servers and 100,000 tasks. Python ran the experiments and plotted the simulation data (Pylab). The balls and bins model was utilized to simulate distributing tasks[9]. The probability of overloaded server and underloaded server were theoretically calculated and simulated. Then, load balancing with one choice and two-choice were simulated and compared. Consistent hashing for stateful task distribution was discussed. Also, consistent hashing with replicas and with uneven load were explored, simulated, and compared.

2.4 Probability a Server is Overloaded

In this section, the probability that a server was overloaded and underloaded was derived. This showed that the probabilities of overloading and underloading **exponentially decayed** as the load increased. A server is overloaded if its load was higher than the average load (e.g. multiplier of $(1+\alpha)$). Chernoff bounds were used to estimate the probability that a given server was overloaded[9].

Let X_i represent the *i-th* task that is allocated on a given server if $X_i = 1$, otherwise $X_i = 0$.
The load on a given server is defined by *X*:

$$X = \sum_{i=1}^{n} X_i$$

$$\mu = E[X]$$

Using Chernoff bounds[8], the probability that X is greater than $1 + \alpha$ times μ is bounded:

$$P(X > (1+\alpha) \times \mu) < [\frac{e^{\alpha}}{(1+\alpha)^{(1+\alpha)}}]^{\mu}$$

In the experiment, $\mu = 100$, so it is substituted into the equation:

$$P(X > (1+\alpha) \times 100) < [\frac{e^{\alpha}}{(1+\alpha)^{(1+\alpha)}}]^{100}$$

The above math theory and the simulation results were shown in Figure 1.

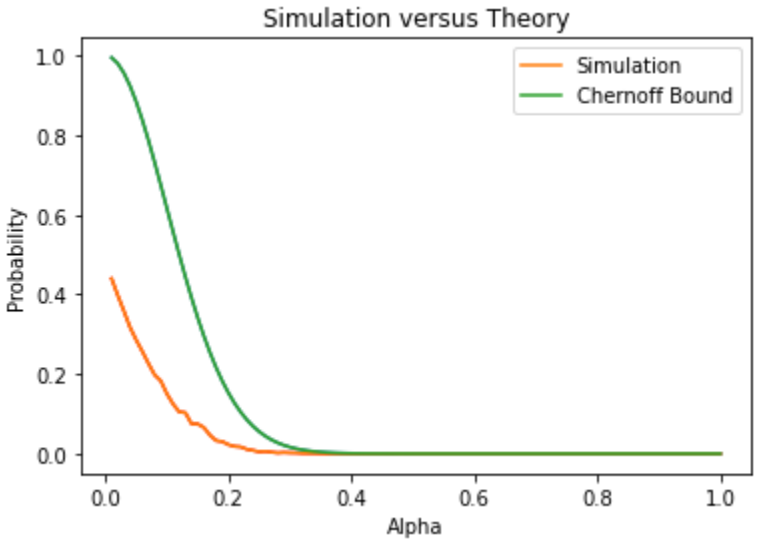


Figure 1. Simulation versus Chernoff Bounds, above bound.

In the simulation, the exact same tasks and server inputs were used. This bound remains true, since there is **no** point at which the orange line intersects the green line. In reality, the probability is actually closer to the x-axis, but a bound is just an inequality.

2.5 Probability a Server is Underloaded

Similar to how Chernoff bounds can be applied for overloaded servers, it can account for underloaded servers as well. Below is the probability *X* will be smaller than $(1-\alpha)\mu$:

$$P(X < (1-\alpha)\mu) < [\frac{e^{-\alpha}}{(1-\alpha)^{(1-\alpha)}}]^{\mu}$$

Since the expected load is 100, it is substituted for μ:

$$P(X < (1-\alpha)100) < [\frac{e^{-\alpha}}{(1-\alpha)^{(1-\alpha)}}]^{100}$$

Since the distribution of tasks is uniform, symmetry between both bounds is expected. If the amount of extra tasks on overloaded servers is removed, it must be enough to fill the underloaded servers to exactly the expectation of the load. That is because in a perfect scenario it must have a consistent load of 100 tasks. Below is the graph of the simulation shown in Figure 2.

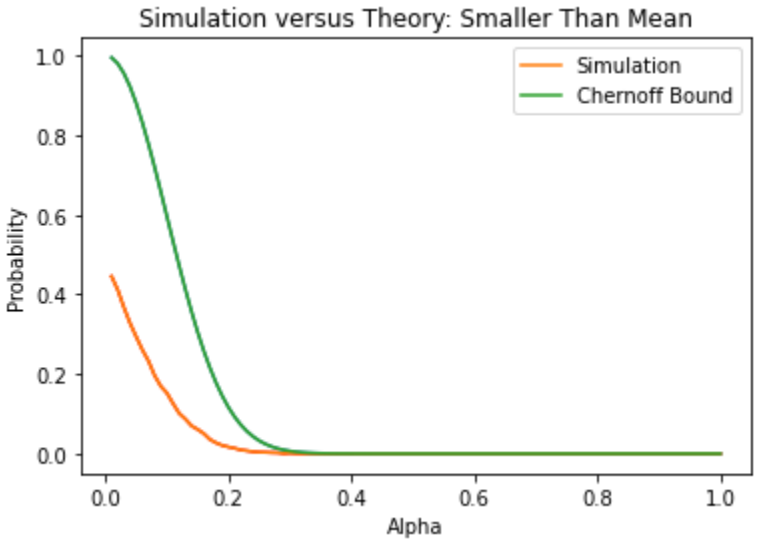


Figure 2. Simulation versus Chernoff Bounds, below bound.

Identical probabilities with the overloaded probability and underloaded probability were clearly shown. The Chernoff bound clearly holds since the orange line never intersects the green line.

2.6 Load Balancing with Two-Choice Model

In the section above, the load distribution on servers was studied, and the load was mostly concentrated around the expected values. The probabilities of overloading and underloading servers exponentially decays as the load moves aways from the expectation. Whereas these are desirable properties in theory, it can further be improved in practice. In this section, the two choice model and its comparison with one choice load balancing were discussed.

Load Balancing with One Random Choice

The simplest way to distribute tasks to servers is to randomly route a task, assuming it is a stateless server. A hash function maps an input to a hashed range in a way that is uniformly random, similar to a math function where you plug in an x-value to obtain the corresponding y-value. Also, the random.randint() function in Python that randomly selects a number from a range that corresponds to a server was used. This is a simple and powerful method because it does not require any additional information to route a task to a server. Furthermore, if X represents the number of tasks:

$$X \sim Pois(\lambda)$$

The Poisson distribution represents the probability that a number of events will arrive in a fixed interval of time. After running load balancing with one random choice, an average load graph was shown in Figure 3.

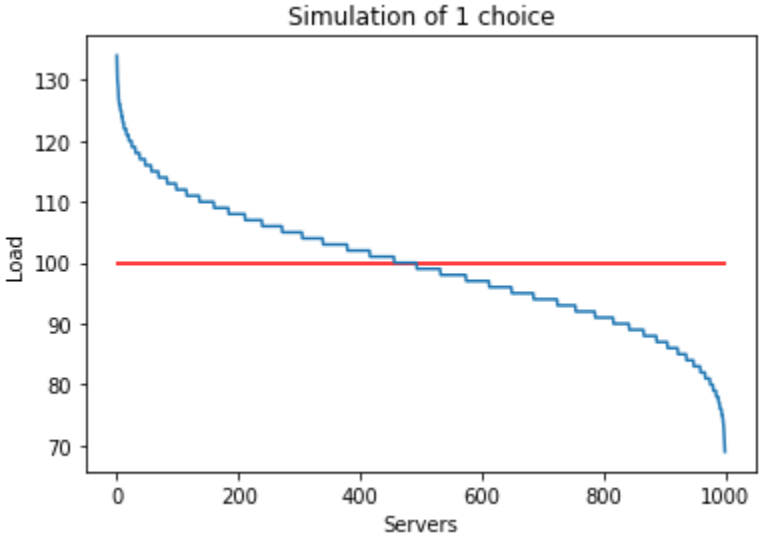


Figure 3. Simulation of 1 choice load balancing.

Even with this graph, it can be inferred that there was a diverse range of loads, which was not the most efficient way of balancing the load. Also, all of the graphs in this paper were sorted by decreasing for comparison purposes.

Load Balancing with Two Random Choices

In this section, the powerful tool of two choices was explored[9]. This acts in a very similar way to the randomized model, except the computer had two choices for each arriving task. After randomly selecting two servers, the server with fewer tasks accepted the task. It is expected that the server loads should distribute better, since the idle servers get filled up. Below was a graph of the data collected for two random choices that is **zoomed** up in Figure 4.

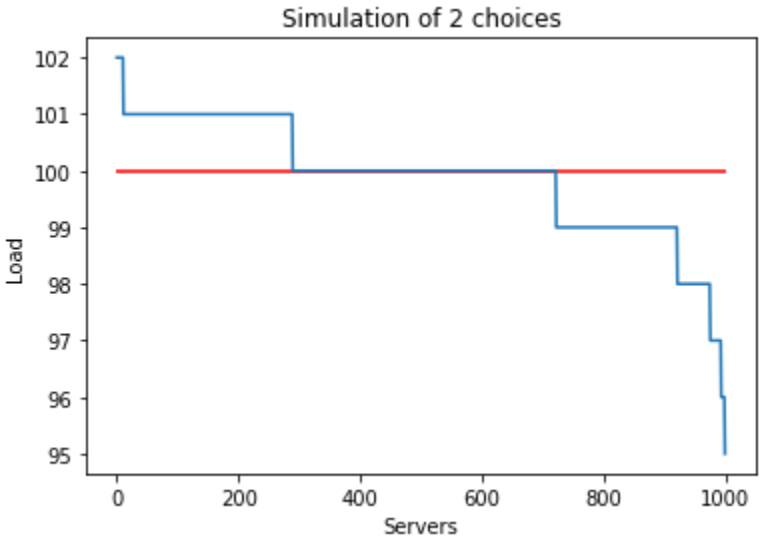


Figure 4. Simulation of two choices load balancing.

Now, the two algorithms were plotted the exact same plot in Figure 5.

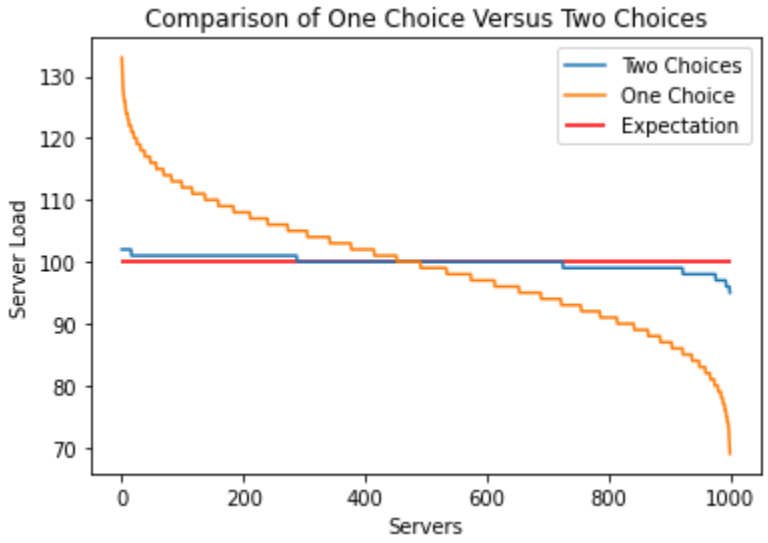


Figure 5. Load balancing comparison of one choice and two choices.

This was clearly a significantly more efficient way of load balancing and visually observing, it was definitely clear. By increasing one more choice, it was extremely close to the optimal solution, where every single server had exactly the same load.

Comparison of One versus Two Choices

Table 1 shows the metrics compared between one and two choices.

Metrics	**One Choice**	**Two Choices**	**Comparison Notes**
Task Range	[69,134]	[95, 102]	9.29 times smaller range.
Expected Tasks per Server	100	100	Same
Standard Deviation	10.03	0.97	**10.37 times smaller standard deviation**.
Variance	99.56	1.020	97.61 times smaller variance.
Server Task Count Below Expectation	508	278	230 less servers below average.
Server Task Count at Expectation	38	433	395 servers more at expectation.
Server Task Count Above Expectation	454	289	165 less servers above average.

Table 1. Comparisons of one choice versus two choices methods.

From the data, by increasing one random choice, it allowed the task count to be 10.37 times closer to the mean than using one choice. Furthermore, it shrank the task range by 9.29 times. Also, 395 more servers were at the expectation. Let's assume servers were overloaded or idle if α = 0.2. Each server can serve up to 120 tasks, however it was allowed that the tasks may be slightly above from 120 tasks because the probability was so small of an event. Since 100,000 tasks were used, it can be used to calculate the servers necessary:

$$100000\ tasks//120\ max\ load = 833\ servers$$

Below was the simulation with 833 servers using two choices shown in Figure 6.

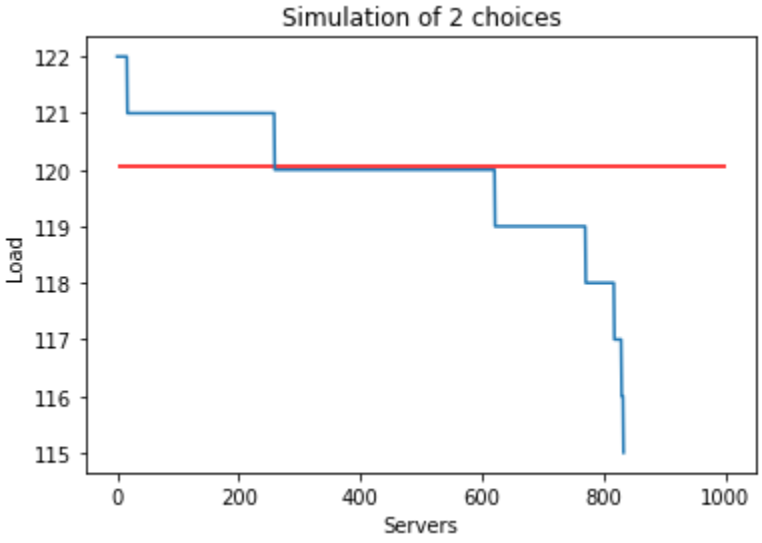


Figure 6. Simulation of two choices with 833 servers.

Even though it was above 120 tasks, it can still be considered since it was barely above 120 tasks and is considered an **extremely** rare event. This simulation of 833 servers ran with a total of **16.7% less servers** than using the randomized algorithm. Using the 203 terawatts that was

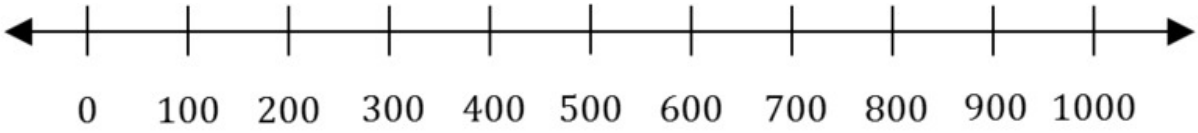
mentioned earlier[1], this algorithm saved 33.901 terawatts of energy using the two-choice algorithm[4]. That was 24,025,045 metric tons of CO_2 saved from being emitted. Or enough for **4,363,973** houses to be powered for a single year[4].

2.7 Consistent Hashing For Stateful Task Distribution

Sometimes randomly assigning tasks to servers cannot work, which is the case with stateful servers. A stateful server stores data or metadata required for serving **particular** requests, because no single server can hold the whole dataset. For example, to check your email, a given email server has to know your data location to retrieve your list of emails to see if you have any emails. That **specific** server has access to where data is stored because each server can only hold a partition of data. So, there must be a way that will consistently find that same server that will look up your email. Consistent hashing is reliable even though server configuration may change. It is only necessary to run **one** simulation for each problem, since it returns the same result every time because of its reliability. For this, a hash function is needed to distribute the load evenly as above. The Python hash library *md5* was used, which accepts both strings and numbers[10]. The hash function was implemented below:

$$hash(value) = md5.hash(value) \,\%\, 1000$$

By using the modulo function, the hash function only accepts values between 0 to 1000. Imagine a number line below that represents the range of hash values:



Now, each server needs to be hashed by its server number, to determine the ranges the servers will be accepting tasks. Below is the server placement on the number line using the hash function:

$$hash(1) = 88.411$$

$$hash(2) = 479.212$$

$$hash(3) = 880.883$$

Below the dots represent approximately where the servers place on the range shown in Figure 7.

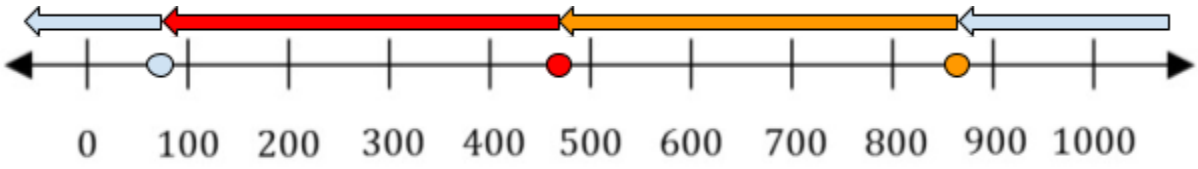


Figure 7. Number line with 3 servers consistent hashing.

The blue range corresponds with server 1, the red range corresponds with server 2, and the orange range corresponds with server 3. When a task gets hashed onto the number line, the server that is responsible for the range will serve the task.

The simulation below illustrated the load distribution with consistent hashing shown in Figure 8.

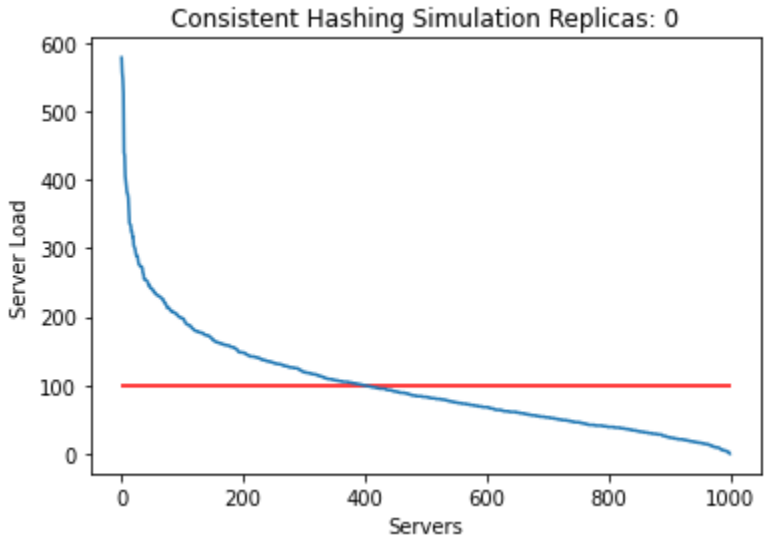


Figure 8. Consistent hashing with only servers.

This simulation has an extreme overload on some servers, about **6 times** the optimal solution. This was due to hotspots since one particular server served a significantly large range. In the next section, a more efficient way of consistent hashing was explored.

Consistent Hashing with Replicas

The problem with the consistent hashing that was used in the previous section was that the server load will skew to the input distribution. Concretely, while the hash function tries to distribute the task uniformly, the load distribution might still be skewed. To address this problem, server replication is introduced. Replicas allow servers to split the value range further by hashing more ranges on the number line[8,9]. Replicas are another way to add more ranges for a server. This means each server can serve a number of smaller hashed ranges, hence the load on each server is more likely to be even. Replica servers are now hashed, using the formula below:

$$hash(string(M\ Replica\ N))$$

Where *M* represents the server number and *N* represents the replica number. Let's update the number line above for two replicas for each server shown in Figure 9.

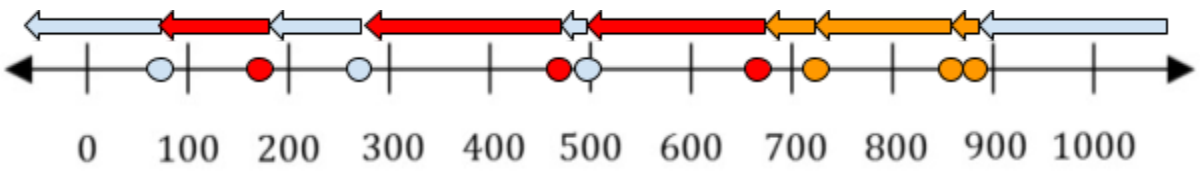


Figure 9. Number line with 3 machines and two replicas consistent hashing.

The number line shows that the ranges are significantly shrunken. Each servers' range is spread out, shrunken, and is not consecutive as you increase more replicas. For example, server 1 and server 2 now have interlaced ranges. This decreases the chance of large loads as replicas increase. In the simulation, 100,000 tasks, 1,000 servers, and up to 100 replicas were used. Below was the simulation with 10 replicas shown in Figure 10.

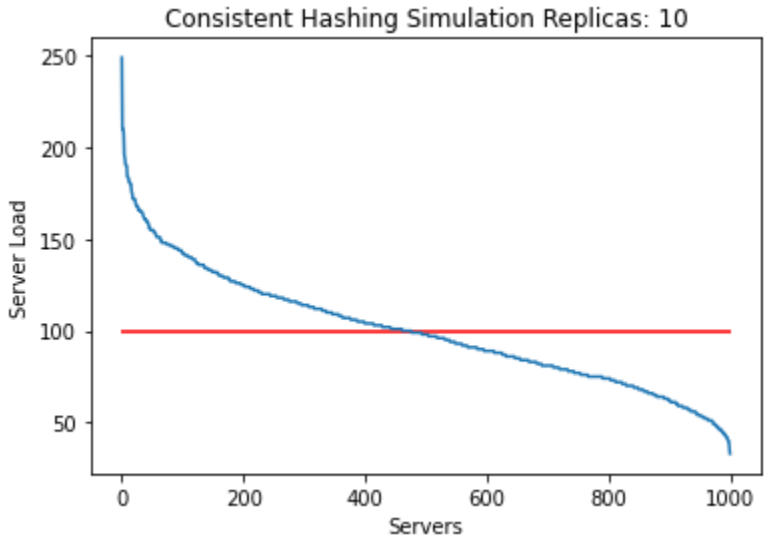


Figure 10. Consistent hashing with 10 replicas.

This is a significant improvement over the no replicas simulation. Below was the simulation with 100 replicas shown in Figure 11.

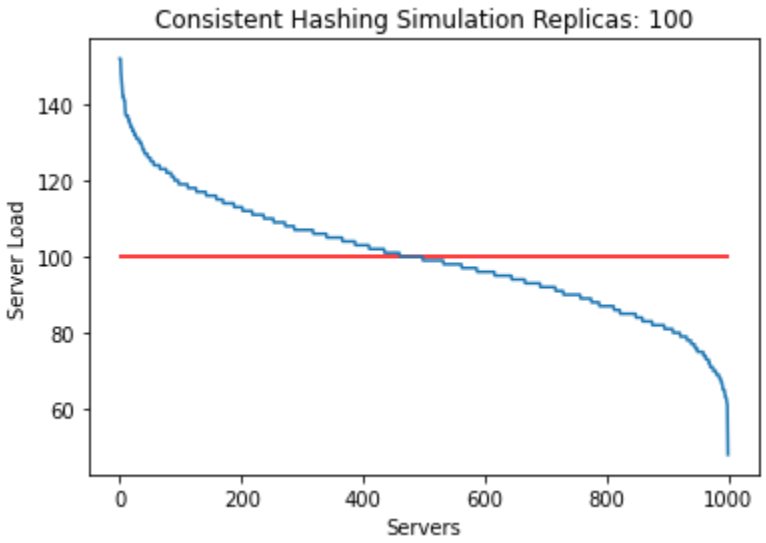


Figure 11. Consistent hashing with 100 replicas.

Below are the results of this simulation shown in Table 2.

Metrics	No Replicas	10 Replicas	100 Replicas	Comparison Notes
Task Range	[0,579]	[33,249]	[48,152]	
Range Length	579	216	104	Range decreased by 5.57 times.
Expected Tasks per Server	100	100	100	Same.
Standard Deviation	76.62	31.15	15.2636	Standard deviation decreased by **5.02** times.
Variance	5870.75	970.60	232.976	Variance decreased by 25.2 times.
Server Task Count Below Expectation	599	524	502	Server task count below expectation decreased by 97 servers.
Server Task Count at Expectation	5	15	38	Server task count at expectation increased by 33 servers.
Server Task Count Above Expectation	396	461	460	Server task count above expectation increased by 64 servers.

Table 2. Comparison of consistent hashing with no replicas, 10 replicas, and 100 replicas.

Here are all of the methods plotted on a single graph shown in Figure 12.

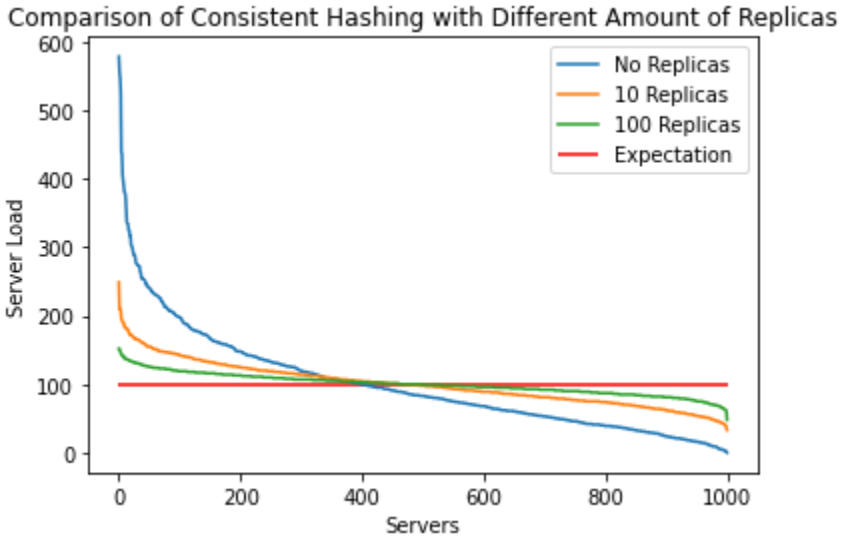


Figure 12. Consistent hashing plot comparison with no replicas, 10 replicas, and 100 replicas.

In the simulation, it was found that there was an extreme decrease of 5.02 times of the standard deviation, which is a significant improvement compared to no replicas. Also, the range is shrunken by 5.57 times which is significant. Seeing the graph above, an extremely small number of extreme outliers can be seen. As well as the fact that increasing the replicas showed less skewness to the left.

3. Conclusion

To conclude, in this study the environmental impact of data centers and how it can improve data center efficiency was researched. Using the understanding of 203 terawatt hours of the data center industry[1], it was concluded that data centers use lots of energy. In addition, this study used Chernoff Bounds to establish the upper bound on the probability of underloaded or overloaded servers. Methods like consistent hashing to solve the problem of stateful servers were utilized and with the finalized algorithm including 100 replicas it proved to be 5.02 times more efficient instead of no replicas. Furthermore, important math concepts were explained using probability distributions to understand the balls and bins model. For the stateless servers, a 10.37 times improvement from the two-choice algorithm to the randomized model was found. Also, approximately 16.70% of energy was saved by reducing the server count on the efficient method to still be under the overload limit.

For future research, the model of load balancing will be enhanced for modeling different types of workloads. And the model of load balancing can be used for optimizing network package routing and traffic control. In summary, load balancing for stateful and stateless servers has many real world applications and can help make data centers more efficient and therefore reduce the environmental impact of data centers.

References

1. Masanet, E., Shehabi, A., Lei, N., Smith, S., & Koomey, J. (2020, March 20). Retrieved February 13, 2022, from https://datacenters.lbl.gov/sites/default/files/Masanet_et_al_Science_2020.full_.pdf

2. Schlichting, Alexander. Data Center Energy Efficiency Technologies and Methodologies. 0715D190EP, MITRE, Nov. 2015, p. 81, https://apps.dtic.mil/sti/pdfs/AD1014836.pdf.

3. *Department of Defense Implementation of the Federal Data Center Consolidation Initiative: Implications for Federal Information Technology Reform Management*. R42604, 23 Apr. 2013, p. 24, https://crsreports.congress.gov/product/pdf/R/R42604/9.

4. US EPA, O. (2015, August 28). *Greenhouse gas equivalencies calculator* [Data and Tools]. United States Environmental Protection Agency. https://www.epa.gov/energy/greenhouse-gas-equivalencies-calculator

5. Berti, D., & Snyder, R. (2021, April 20). *Data center market drives local vacant land prices near $1 million per acre*. Prince William Times. Retrieved February 13, 2022, from https://www.princewilliamtimes.com/news/data-center-market-drives-local-vacant-land-prices-near-1-million-per-acre/article_c8f1c0a2-a211-11eb-b916-d32646d62ed0.html

6. *Google apps: Energy efficiency in the cloud*. (n.d.). Retrieved February 12, 2022, from https://static.googleusercontent.com/media/www.google.com/en/us/green/pdf/google-apps.pdf

7. Google. (n.d.). *Efficiency – data centers*. Google. Retrieved February 12, 2022, from https://www.google.com/about/datacenters/efficiency/#:~:text=Most%20data%20centers%20use%20almost,serving%20Google%20searches%20and%20products

8. Adya, A., Myers, D., Howell, J., Elson, J., Meek, C., Khemani, V., Fulger, S., Gu, P., Bhuvanagiri, L., Hunter, J., Peon, R., Kai, L., Shraer, A., Merchant, A., & Lev-Ari, K. (2016, November 2). *Slicer: Auto-sharding for datacenter applications - usenix*. USENIX. Retrieved February 12, 2022, from https://www.usenix.org/system/files/conference/osdi16/osdi16-adya.pdf

9. Mitzenmacher, M., & Upfal, E. (2005). *Probability and computing: Randomized algorithms and probabilistic analysis*. Cambridge: Cambridge University Press.

10. Nielsen, M. (2009, June/July). Consistent hashing. Retrieved February 12, 2022, from https://michaelnielsen.org/blog/consistent-hashing/

11. Karger, D., Lehman, E., Leighton, T., Levine, M., Lewin, D., & Panigrahy, R. (1998). *Consistent hashing and random trees ... - princeton university*. Princeton. Retrieved February 12, 2022, from https://www.cs.princeton.edu/courses/archive/fall09/cos518/papers/chash.pdf